Automation Testing and Playwright Fundamentals

A foundational introduction to browser automation, Playwright architecture, comparison with Selenium, and the 20-day learning roadmap.

COURSE SPECIFICATIONS

- Track: Modern Browser Automation (JavaScript)
- Level: Beginner
- Target Audience: Beginners, Manual QA, Developers
- Prerequisites: Basic computer knowledge. No previous Playwright experience required.
- Core Tooling: Node.js, npm, VS Code, Playwright Test
- Language: JavaScript (Pure JS examples)

Day 1 Learning Objectives

- Understand software test automation and identify when and why teams automate tests
- Evaluate trade-offs between manual testing and automated test execution
- Identify high-ROI test scenarios for automation vs tests best suited for manual exploration
- Explain Playwright fundamentals: browser engines (Chromium, Firefox, WebKit), headless execution, and auto-waiting
- Compare Playwright with Selenium objectively across architecture, synchronization, and ecosystem
- Understand the 9-stage automation testing workflow and the distinction between Browser, Context, and Page
- Familiarize with the core developer tooling (Node.js, npm, VS Code, JavaScript, Git) and the 20-day course roadmap

Software testing is the systematic process of verifying that an application functions according to business specifications, protects user security, and delivers a reliable user experience.

What Test Automation Means

Test automation is the practice of using software tools and scripts to execute predefined test cases, simulate human user interactions, compare actual outcomes with expected outcomes, and report pass/fail verdicts automatically without manual human intervention.

Why Companies Automate Tests

1. **Speed & Rapid Feedback: Automated test suites can run in minutes during Continuous Integration (CI), giving developers immediate feedback on whether new code broke existing features.**
2. **Reliability & Consistency: Machines execute the exact steps and test data with mathematical precision, eliminating human fatigue and accidental oversights during repetitive checks.**
3. **High Regression Coverage: As an application grows from 10 to 500 features, executing all regression tests manually becomes impossible within a 2-week sprint. Automation runs hundreds of regression tests overnight or on every git commit.**
4. **Cost Efficiency Over Time: While automation has an upfront cost (script writing and environment setup), it drastically reduces cost-per-execution over months of releases.**

Automation Does Not Replace All Manual Testing

A common industry misconception is that automation eliminates the need for manual testers. In reality, **automated tests only verify what you explicitly instruct them to verify. They cannot notice visual awkwardness, judge intuitive user experience, or explore unexpected edge cases.**

What to Automate vs What to Test Manually

- **Prime Candidates for Automation:**
  - **Repetitive regression tests executed on every release build.**
  - **Smoke / Sanity tests verifying critical paths (e.g. user login, checkout payment).**
  - **Data-driven tests requiring hundreds of diverse input combinations.**
  - **Complex multi-step mathematical calculations (e.g. taxes, discounts, shipping fees).**
  - **Cross-browser compatibility tests across Chromium, Firefox, and WebKit.**
- **Best Suited for Manual or Exploratory Testing:**
  - Newly built features with unstable, frequently changing UI designs.
  - One-time or ad-hoc validation tests.
  - User experience (UX), visual appeal, font clarity, and subjective usability.
  - Exploratory testing where human curiosity uncovers unexpected boundary conditions.

The Role of an Automation QA Engineer

An automation QA engineer is not merely a "coder." You analyze requirements, design reliable test scenarios, architect maintainable test frameworks, investigate intermittent test failures, and collaborate with developers to uphold software quality gates.

Manual Testing vs Automation Testing Comparison

Dimension	Manual Testing	Automation Testing (Playwright)
Speed & Execution	Human speed; minutes to hours per test	Machine speed; seconds per test suite
Repetition Cost	High continuous human cost per run	Near-zero cost per run after script setup
Regression Feasibility	Difficult to execute exhaustively each sprint	Runs hundreds of tests overnight or on git push
Best Suited For	UX, visual appeal, exploratory tests	Repetitive flows, calculations, cross-browser
Defect Detection	Finds unexpected edge cases & design flaws	Finds regressions and deviations from specs

Playwright is an open-source, modern browser automation framework developed and maintained by Microsoft. It enables reliable end-to-end (E2E) testing, component testing, and API testing for modern web applications.

Understanding Browser Automation

Browser automation means controlling a real web browser programmatically via code. Your script tells the browser: "Navigate to localhost:5173", "Type an email", "Click the Sign In button", and "Verify that the dashboard URL loads."

What Is Playwright Test?

Playwright comes with **Playwright Test, a comprehensive, enterprise-grade test runner specifically built for end-to-end automation. It provides:**

- **Native test runner with parallel execution out of the box.**
- **Built-in Web-First Assertions with smart polling.**
- **Automatic browser driver management (Chromium, Firefox, WebKit).**
- **Rich debugging tools: Playwright UI mode, VS Code extension, and Trace Viewer.**
- **HTML reporting with video recordings, screenshots, and DOM snapshots.**

Supported Browser Engines

Unlike older tools that relied on third-party browser drivers, Playwright natively supports the three core rendering engines powering over 99% of web traffic:

1. **Chromium:** Powers Google Chrome, Microsoft Edge, Brave, and Opera.**
2. **Firefox:** Powers Mozilla Firefox.**
3. **WebKit:** Powers Apple Safari (iOS and macOS).**

Cross-Browser Testing: You write your test script once in JavaScript, and Playwright can execute it across Chromium, Firefox, and WebKit simultaneously on Linux, macOS, and Windows.

Headless vs Headed Execution

- **Headed Mode:** The browser opens visibly with a graphical user interface window on your monitor. You can see the clicks, navigation, and typing in real time. Great for local debugging!**
- **Headless Mode:** The browser runs silently in memory without opening a physical graphical window. This is significantly faster, uses fewer system resources, and is the standard mode used in CI/CD build servers (like GitHub Actions).**

Auto-Waiting: Why Playwright Tests Are Resilient

In legacy frameworks, scripts frequently failed because the script tried to click a button before the JavaScript on the page finished loading.

Playwright solves this fundamentally through **Auto-Waiting**: before performing an action (such as `click()` or `fill()`), Playwright automatically performs actionability checks to verify that the element is:

1. Attached to the DOM.
2. Visible (not hidden by CSS or opacity 0).
3. Stable (not animating or sliding across the screen).
4. Enabled (not disabled).
5. Ready to receive pointer events (not blocked by a loading overlay).

Playwright vs The Application Under Test

It is vital to distinguish between:

- **The Application Under Test (AUT):** The website you are testing (e.g. RootAccess web portal built with React).**
- **The Automation Tooling (Playwright):** The independent test framework written in JavaScript running in Node.js that drives the browser and assesses the AUT.**

Note on API Testing: While Playwright is famous for browser UI automation, it also natively provides an HTTP client (`request`) for lightning-fast API testing, which we will explore later in the course!

```
Results & Reports  <% % % % %  Assertions Verified   <% % % % % %   Web Application
```

TOPIC 3: PLAYWRIGHT VS SELENIUM

When entering test automation, one of the first questions engineers ask is: *"How does Playwright compare to Selenium?"*

Selenium has been the undisputed industry standard for over 15 years. It is an open-source, W3C-standardized tool maintained by a massive global community. Playwright was introduced more recently to solve specific modern architectural challenges of Single-Page Applications (SPAs) and dynamic JavaScript frameworks.

Balanced Comparison Matrix

Neither tool is "always better." The right choice depends on the project's technology stack, legacy investments, and organizational goals.

Why Selenium Remains Widely Used

Selenium is not "dead" or obsolete. Thousands of enterprise banking, healthcare, and telecom systems have mature test suites with millions of lines of Selenium Java code running in production. Furthermore, Selenium WebDriver is an official W3C web standard.

However, for modern dynamic web applications (React, Vue, Angular, Next.js), Playwright offers faster test execution, zero-configuration setup, native auto-waiting, and far fewer flaky test failures.

Dimension	Playwright	Selenium WebDriver
Architecture	Direct WebSocket via Chrome DevTools Protocol	HTTP REST via intermediary drivers (ChromeDriver)
Speed & Overhead	Fast; persistent single socket connection	Moderate; HTTP request/response for each command
Waiting Strategy	Native auto-waiting on all actions	Requires manual WebDriverWait / ExpectedConditions
Browser Engines	Bundled Chromium, Firefox, WebKit	External driver binaries for Chrome, Gecko, Safari
Test Runner	Integrated @playwright/test runner	Requires 3rd-party runners (Mocha, Jest, JUnit)
Ecosystem / Age	Modern, rapid adoption by Microsoft	15+ years maturity, W3C official standard

TOPIC 4: UNDERSTANDING THE AUTOMATION TESTING WORKFLOW

Writing automated tests is not about jumping straight into code. Professional QA follows a disciplined 9-stage engineering workflow:

The 9-Stage Automation Testing Workflow

1. **Understand Application Requirements: Thoroughly read user stories, acceptance criteria, and business logic.**
2. **Identify Test Scenarios: Determine the complete set of positive, negative, and edge-case user journeys.**
3. **Decide Automation Candidates: Select high-value, repetitive regression paths; leave one-off UX checks for manual exploration.**
4. **Prepare Test Environment & Test Data: Ensure staging databases have clean user accounts, test products, and API credentials.**
5. **Write Automated Test Scripts: Use clean JavaScript, maintainable locators, and clear assertions.**
6. **Execute Tests: Run tests locally in headed mode or against headless CI/CD test grids.**
7. **Review Assertions, Failures & Reports: Inspect HTML reports, screenshots, and logs to assess test outcomes.**
8. **Investigate Failures: Distinguish true application bugs from script locator defects or environment network glitches.**
9. **Maintain Test Suites: Update selectors and assertions as product design evolves over sprints.**

Core Concepts: Browser vs Browser Context vs Page

Understanding this 3-tier hierarchy is the foundation of Playwright architecture:

1. **Browser:** An instance of Chromium, Firefox, or WebKit. Launching a browser takes 1–2 seconds, so Playwright launches it once per worker thread.
2. **Browser Context:** A lightweight, completely isolated incognito browser session. Each context has its own cookies, localStorage, and cache. Creating a context takes only 2 milliseconds! This allows you to test two different users (e.g. Student vs Admin) in the same test without interference.
3. **Page:** A single tab or window inside a Browser Context where your script navigates, clicks, and asserts elements.

Conceptual Example: Automating a Login Regression Scenario

On Day 1, you do not need to write working code yet. But here is what a clean, modern Playwright test written in **JavaScript looks like conceptually:**

Day 3	Day 3 — Environment Setup and First Playwright Test	Installing Node.js, initializing Playwright in a project, and executing your first headed test
Day 4	Day 4 — Locators and Element Identification	Accessible locators (getByRole, getByLabel, getByText), CSS selectors, and filtering
Day 5	Day 5 — Actions, Assertions and Web Forms	Clicking, typing, radio buttons, checkboxes, dropdowns, and web-first assertions
Day 6	Day 6 — Waiting Strategies and Reliable Tests	Understanding auto-waiting, explicit waiting, eliminating flakiness, and timeouts
Day 7	Day 7 — Navigation, Tabs, Frames and Dialogs	Handling multiple tabs, iframe elements, JavaScript alerts, popups, and downloads
Day 8	Day 8 — Page Object Model (POM) Architecture	Structuring enterprise-grade test suites using clean Page Object Model architecture
Day 9	Day 9 — API Testing and Network Interception	Testing REST APIs using Playwright request context, validating status codes and JSON payloads
Day 10	Day 10 — Test Fixtures, Hooks & Organisation	beforeEach, afterEach, describe blocks, custom fixtures, and test structuring
Day 11	Day 11 — Data-Driven Testing with JSON & CSV	Reading JSON test fixtures, parameterized tests, and environmental variables
Day 12	Day 12 — Authentication and Storage State Reuse	Saving and reusing authentication state (storageState) to skip repeated logins
Day 13	Day 13 — Visual Regression Testing & Screenshots	Pixel diffing, toHaveScreenshot, snapshot baselines, and layout verification
Day 14	Day 14 — CI/CD Pipeline Integration with GitHub Actions	Configuring workers, cross-browser matrices, and automating tests via GitHub Actions
Day 15	Day 15 — Debugging, Trace Viewer & Video	Playwright UI mode, step-by-step inspector, Trace Viewer, screenshots, and HTML reports
Day 16	Day 16 — AI-Assisted Test Automation & Prompting	Prompting LLMs with DOM context, generating assertions, and evaluating test reliability
Day 17	Day 17 — AI Self-Healing Locators & Root Cause Analysis	Detecting UI mutations, fallback locator strategies, and AI failure diagnostics
Day 18	Day 18 — Synthetic Test Data & Edge Case Generation	Leveraging AI for diverse boundary data, schema fuzzing, and security edge cases
Day 19	Day 19 — Capstone E-Commerce End-to-End Pipeline	Production-ready e-commerce checkout regression pipeline with multi-browser coverage
Day 20	Day 20 — Real-World Portfolio & Certification Exam	Portfolio defense, comprehensive practical exam, and industry certification.

Confirmation.

Task: Populate at least 10 scenarios in your interactive workspace table, classify each as Automation Candidate or Manual/Exploratory, assign priority, state your rationale, identify 3 high-priority regression scenarios, and explain 1 risk of unreliable automated tests.

Starter Example Scenarios (From Course Assignment Table):

TC-ECOMM-01	[Automation - High] Verify registered user logs in successfully with valid email and password and redirects to account dashboard Reason: Critical path executed on every regression cycle; predictable inputs and outputs with high repetition value.
TC-ECOMM-02	[Manual / Exploratory - Medium] Evaluate usability and relevance of search auto-suggestions when typing ambiguous keywords or partial brand names Reason: Requires human evaluation of relevance and search layout; better evaluated through human exploratory testing.

Q1. What is software test automation?

[Difficulty: Beginner] What Interviewer Evaluates: Assesses foundational knowledge of automation principles and business benefits.

Suggested Answer: Software test automation is the practice of using specialized software tools and code scripts to execute test cases, simulate user actions, compare actual outcomes with expected outcomes, and report defects automatically without manual intervention. It increases release velocity, improves regression coverage, and provides rapid feedback to engineering teams.

Follow-Up Question: "Does automating a test make it inherently better than executing it manually?"

Q2. What is the difference between manual testing and automation testing?

[Difficulty: Beginner] What Interviewer Evaluates: Assesses balanced understanding of when human testing is required vs when automation is optimal.

Suggested Answer: Manual testing is conducted by human testers who interact directly with the application to evaluate usability, design aesthetics, and exploratory edge cases. Automation testing uses scripts to execute repetitive, predefined steps at high speed. Manual testing is best for new, exploratory, and UX testing; automation is best for repetitive regression suites, smoke tests, and data-driven calculations.

Follow-Up Question: "Can an automation test suite catch visual misalignment or poor UX navigation on its own?"

Q3. What is Playwright?

[Difficulty: Beginner] What Interviewer Evaluates: Assesses clarity on Playwright architecture, developer, and core capabilities.

Suggested Answer: Playwright is an open-source browser automation framework developed by Microsoft for reliable end-to-end web testing. It controls modern browser engines (Chromium, Firefox, WebKit) via direct browser protocol connections, features native auto-waiting to eliminate flakiness, supports cross-browser and mobile-viewport emulation, and includes built-in test runners and trace debugging tools.

Follow-Up Question: "Which programming languages does Playwright officially support?"

Q4. Which browser engines does Playwright support?

[Difficulty: Beginner] What Interviewer Evaluates: Assesses technical accuracy regarding rendering engines vs commercial browser brands.

Suggested Answer: Playwright supports three major browser rendering engines: 1. Chromium (powers Google Chrome, Microsoft Edge, Opera), 2. Firefox (powers Mozilla Firefox), and 3. WebKit (powers Apple Safari on iOS and macOS). Playwright bundles and manages these engines directly without requiring external driver binaries.

Follow-Up Question: "How does testing on WebKit in Playwright differ from testing on actual macOS Safari?"

Q5. What is Playwright Test?

[Difficulty: Beginner] What Interviewer Evaluates: Assesses understanding of Playwright's built-in test runner vs standalone automation library.

Suggested Answer: Playwright Test (@playwright/test) is Playwright's comprehensive, dedicated test runner. It provides native test declarations (`test()`), describe blocks, web-first assertions (`expect()`), parallel execution across multiple worker processes, test fixtures, screenshot/video capture on failure, and rich HTML reporting.

Follow-Up Question: "Can Playwright be used with external runners like Jest or Mocha if an enterprise already uses them?"

Q6. What is the difference between headed and headless execution?

[Difficulty: Beginner] What Interviewer Evaluates: Assesses execution environment understanding and performance optimization.

Suggested Answer: In headed mode, the browser opens a visible graphical user interface window on screen, allowing the engineer to visually observe clicks, page loads, and text input in real time. In headless mode, the browser executes silently in system memory without a GUI. Headless mode runs significantly faster, uses less CPU/memory, and is the standard mode used in CI/CD pipeline automation.

Follow-Up Question: "Why might a test pass in headed mode on your laptop but fail in headless mode on a Linux CI server?"

Q7. What is regression testing?

[Difficulty: Beginner] What Interviewer Evaluates: Assesses understanding of software change impact and test suite selection.

Suggested Answer: Regression testing is the selective execution of test cases to verify that recent code changes, bug fixes, or enhancements have not introduced new defects or broken existing, unchanged functionality. Because regression suites must be executed repeatedly before every release, they are the primary candidate for automation.

Follow-Up Question: "How do QA teams choose between executing a full regression suite vs a smoke test suite?"

Q8. What is the difference between Playwright and Selenium?

[Difficulty: Intermediate] What Interviewer Evaluates: Evaluates architectural knowledge and objective comparison of automation tools.

Suggested Answer: Selenium communicates with browser drivers (ChromeDriver, geckodriver) using HTTP REST commands over the W3C WebDriver specification, requiring external drivers and explicit wait logic. Playwright connects directly to browser engines over a single fast WebSocket connection via Chrome DevTools Protocol, features built-in auto-waiting, bundles Chromium/Firefox/WebKit out of the box, and includes an all-in-one test runner and trace viewer. Selenium has greater 15+ year legacy maturity, while Playwright offers faster execution and lower flakiness for modern SPAs.

Follow-Up Question: "What is an advantage of Selenium over Playwright in enterprise environments?"

Q9. Which test cases should be considered for automation?

[Difficulty: Beginner] What Interviewer Evaluates: Assesses ROI analysis and practical test case automation selection.

Suggested Answer: Good candidates for automation include: 1. Repetitive regression tests executed on every release, 2. Critical user journey smoke tests (e.g. login, payment checkout), 3. Data-driven tests requiring hundreds of permutations, 4. Multi-browser compatibility checks, and 5. Complex mathematical calculation validations. Scenarios that should remain manual include brand-new unstable features, one-off checks, and visual UX/usability evaluations.

Follow-Up Question: "What happens if a team tries to automate 100% of all manual test cases?"

Q10. Does automation testing replace manual testing?

[Difficulty: Beginner] What Interviewer Evaluates: Assesses professional maturity and holistic QA philosophy.

Suggested Answer: No, automation testing does not replace manual testing. Automated tests only verify the specific conditions coded by the engineer; they cannot judge human user experience, notice visual oddities, or conduct exploratory testing to uncover subtle domain bugs. High-performing engineering teams combine automated tests for fast regression feedback with skilled manual testers for exploratory, usability, and edge-case discovery.

Follow-Up Question: "How does automation change the day-to-day responsibilities of a manual QA engineer?"

Q11. Scenario Investigation: You have automated a login test. It passes locally but fails intermittently in the test environment. What would you investigate?

[Difficulty: Scenario-Based] What Interviewer Evaluates: Evaluates systematic debugging methodology, root-cause analysis, and practical troubleshooting ability without jumping to quick assumptions.

Suggested Answer: To investigate an intermittent ("flaky") test failure, I would systematically check 6 key areas:

1. **Test Data State:** Is the test account being modified, locked out, or shared concurrently by other parallel test runs?
2. **Environment & Network Stability:** Is the test staging server slower or experiencing API latency / 504 gateway timeouts compared to localhost?
3. **Locator Reliability:** Does the locator rely on dynamic CSS classes (e.g. hashed class names) or position indices that change when other UI elements render?
4. **Timing & Asynchronous Operations:** Is an animation, modal banner, or third-party script temporarily overlaying the login button?
5. **Session & Cookie Interference:** Are residual cookies or auth tokens from previous test runs polluting the browser context?
6. **Artifact Inspection:** I would inspect Playwright Trace Viewer, failure screenshots, video recordings, and network console logs to observe the exact DOM state at the millisecond of failure.

Follow-Up Question: "Why is adding arbitrary sleep timeouts like `page.waitForTimeout(5000)` a bad fix for this problem?"